

Python Common Services RfExtGeo
Utilities

Table of Contents

Copyright Notice	1
Change History	1
1. Introduction	2
2. Installation	3
3. Streams	4
3.1. Audio Streams	4
3.1.1. Source	4
AudioTone	4
WavReader	4
3.1.2. Destination	4
AudioPlayer	4
WavWriter	5
3.2. Power Spectral Density (PSD) Streams	5
3.2.1. Source	5
3.2.2. Destination	5
PsdPlot	5
4. I/Q Observable and Replica Files	6
4.1. Midas BLUE	6
4.1.1. Fixed Header fields	6
4.1.2. Adjunct Header Fields	6
4.1.3. Data Section	7
4.1.4. Extended Header	7
Sample timestamps	7
Position timestamps	8
Covariance Matrix	9
4.2. Reading and Writing data	10
4.2.1. Writing to File	10
4.2.2. Writing to a byte array	10
4.2.3. Reading from File	10
4.2.4. Reading from a byte array	10
5. Examples	11
5.1. Sensor	11
5.2. Tasker	14
5.2.1. Commands	15
help	15
exit	15
list	15
sensors	15

tune <sensor-id> <tuner-id> <frequency>	15
capability <sensor-id>	16
status <sensor-id>	16
audio play <sensor-id> <frequency> <bandwidth> <modulation>	16
audio stop	16
psd start <sensor-id> <frequency> <bandwidth> [--bins <bins>] [--duration <duration>] ...	16
psd stop	16
iq start <sensor-id> <frequency> <bandwidth>	16
iq stop <task-id>	16
iq list	16
iq dwell <sensor-id> <frequency> <bandwidth> [--duration <duration>] [--start <start time>] [--task-id <task-id>]	16
ping <resource> [--trace] [--hub] [--hub-only]	16
5.2.2. Example session	16
5.3. Configuration	17
5.3.1. Schema	17

Copyright Notice

© 2019-2026 – CROWN OWNED COPYRIGHT. All Rights Reserved.

The copyright of this Software is vested in the Crown and the Software is the property of the Crown.

This Software is released in confidence, to the recipients nominated by the Crown and is for use only for the purposes of the Crown.

This Software shall not be copied or further disclosed except as authorised by the Crown. Any further disclosure shall be under the terms of this legend and any copies shall be the property of the Crown. On completion of the work in connection with which this Software is released the recipient shall return the Software (and any correspondence) to the issuing authority) <<<

Change History

1.0	Initial Version extracted from pycs-core v4.2
1.1	Added support for I/Q archive and observable requests
2.0	Change package name to pycs.rfextgeo. Update to use async clients.
2.1	Revised RTP interfaces
2.2	Add SSL connection support. Use pycs.client decorators for application startup, config and command line params. Install example sensor and tasker as applications.
3.0	Transport layer abstraction

1. Introduction

The `pys-rfextgeo` package contains helper classes related to communicating with other RfExtGeo clients over the Common Services network. This includes support for Audio and RTP streams in RfExtGeo format, and for creating and reading I/Q files in Midas BLUE format.

It contains a simple example sensor, which can be configured with a number of tuners, then accept tune requests, audio and PSD stream task requests, I/Q archive task requests, and I/Q observable requests.

It contains a simple command line tasker, which can display the capability and status of sensors on the network, send tune requests, play audio streams, display a frequency/amplitude graph of a PSD stream, start/stop I/Q archive tasks, and request I/Q dwell files.

2. Installation

pycs requires an installation of python at least version 3.13. The use of virtual environments is recommended.

```
$ python -m venv venv
$ source ./venv/bin/activate
(venv)$ pip install pycs.rfextgeo
```

To use the pycs package interactively, run python inside the virtual environment

```
(venv)$ python
>>> import pycs.rfextgeo
>>> help(pycs.rfextgeo)
Help on package pycs.rfextgeo:
...
```

3. Streams

3.1. Audio Streams

3.1.1. Source

A new audio source stream type can be created by subclassing the `pys.rfextgeo.streams.audio.AudioSourceStream` class. A sub-classes simply needs to implement the asynchronous `run` method, and make calls to `send_samples` as samples become available. Samples should be provided as numpy arrays of PCM audio data. The default encoder is uLaw, which is provided, but other encoders can be defined by subclassing `pys.rfextgeo.streams.audio.Encoder` and passing this to the constructor. This encoder should implement the `encode` method to convert the incoming PCM samples into arrays of bytes. For convenience a `resample` method is provided for when sample rates need to be converted.

The `AudioSourceStream` class automatically handles encoding of the sample data using the supplied encoder, and inserts RTP extension headers as required by the RfExtGeo IDD.

AudioTone

The `AudioTone` class provides an example of an audio source stream which plays a continuous tone of the supplied frequencies.

WavReader

The `WavReader` class provides an example of an audio source stream which reads a `.wav` file and plays it repeatedly.

3.1.2. Destination

A new audio destination stream type can be created by subclassing the `pys.rfextgeo.streams.audio.AudioDestinationStream` class. A sub-class simply need to implement the asynchronous `process_samples` method, and loop over the arrays of PCM samples returned from the `samples` parameter. The decoder is chosen based on the payload type in the RTP header. A uLaw decoder is provided, but other decoders can be registered by adding to the `AudioDestinationStream.decoders` dictionary, indexed on the RTP payload type identifier. A decoder should implement the `decode` method, taking an array of bytes, and convert to a numpy array of PCM samples.

AudioPlayer

The `AudioPlayer` class provides an example of an audio destination stream which plays the incoming audio stream to the sound device.

This class is only available with the 'audio' extra installed, and requires the portaudio-devel package pre-installed on the machine.

WARNING	There currently appears to be an incompatibility between the pyaudio package
----------------	--

and Python 3.10 (see <https://stackoverflow.com/questions/70344884>). If you wish to use the `AudioPlayer` class, then Python 3.9 is recommended until `pyaudio` is updated.

Centos-7

```
$ sudo yum install epel-release
$ sudo yum install portaudio-devel
$ python -m venv venv
$ ./venv/bin/pip install pycs.rfextgeo[audio]
```

WavWriter

The `WavWriter` class provides an example of an audio destination stream which saves the incoming audio stream to a `.wav` file.

3.2. Power Spectral Density (PSD) Streams

3.2.1. Source

A new PSD source can be created by subclassing the `pycs.rfextgeo.streams.psd.PsdSourceStream` class. A sub-class should implement the asynchronous `run` method, and call `send_sample` as FFT samples become available. The frequency, bandwidth, duration and other fields required for the RTP extended headers can be set using properties on the class. Any changes should be made before sending the sample to which they first apply, and the marker bit in the RTP header will be set appropriately.

3.2.2. Destination

A new PSD destination can be created by subclassing the `pycs.rfextgeo.streams.psd.PsdDestinationStream` class. A sub-class should implement the asynchronous `process_samples` method and loop over the FFT samples provided by the `samples` parameter. As each sample is yielded, the frequency, bandwidth and other properties are set appropriately for that sample.

PsdPlot

The `PsdPlot` class provides an example of a PSD destination stream which pops up a window containing a graph of the frequency/amplitude for the FFT samples.

4. I/Q Observable and Replica Files

4.1. Midas BLUE

RfExtGeo defines that its I/Q Observable and Replica files must be in Midas BLUE format. The class `pys.rfextgeo.files.midas.BlueFile` allows Midas BLUE files to be easily created and manipulated to the RfExtGeo standard.

4.1.1. Fixed Header fields

Most of the fields in the Midas BLUE fixed header are either not applicable to the RfExtGeo conop (and required to be fixed at zero), or are meta data fields describing the format and layout of the rest of the file.

Only the `protected` field is available for user use under RfExtGeo, and is exposed as the `.protected` attribute of type `boolean` on the `BlueFile` class.

Additionally, the four header keywords that are have a prescribed meaning and are applicable to RfExtGeo are exposed as `.creator`, `.io`, `.tc_prec`, and `.ver`. Although all the keywords are represented in the header as strings, the TC_PREC keyword represents a floating point value, so the `.tc_prec` attribute is exposed as a `decimal.Decimal` (freely convertible to/from `float`) with 12 decimal places, correctly rounded to the required picosecond accuracy. Note that `datetime.timedelta` cannot be used, as it is limited to microsecond accuracy.

A keyword may be removed by assigning it the value `None` or by calling python's `del` on the attribute. Reading the value of a keyword that is not present will return `None`

Keywords other than those described above have no defined meaning under RfExtGeo, but may be accessed using the `header_keywords` dictionary. Note that the maximum length of the entire keyword field is 92 characters, and any characters over this limit will be truncated.

```
from pys.rfextgeo.files.midas import BlueFile

blue = BlueFile()
blue.protected = False
blue.timecode = 'now'
blue.tc_prec = 1.23e-7
blue.header_keywords['CUSTOM'] = 'value'
```

4.1.2. Adjunct Header Fields

RfExtGeo restricts the Adjunct Header to be of Type 1000, and this is enforced by `pys`. The `xstart` and `xdelta` fields are exposed as attributes on the `BlueFile` class. The `xstart` field is a `datetime.datetime` value, and can be parsed from a string on assignment. The `xunits` field is constrained by RfExtGeo to the constant value `1` and is not exposed to the user.

4.1.3. Data Section

The data section is constrained by RfExtGeo to be either 16 or 32 bit complex integers (data types **CI**, **CL**), however as other data types are commonly seen in the wild, **pycs** allows all other complex data types to be used.

The data is represented on the **BlueFile** class as a **.data** attribute with type **numpy.ndarray**. When reading from a file, the data type of the array will be calculated from the header **format** field, and will be an $n \times 2$ array, with each entry of the first index being a [real,imag] pair. When setting the array data programmatically, the **format** will be calculated from the numpy **dtype** of the array. The dimensions of the array will be ignored, and the data written in memory order. The endianness of the data section is determined by the endianness of the **data_rep** field for incoming data, and by the endianness of the numpy array **dtype** for outgoing data.

4.1.4. Extended Header

RfExtGeo defines a number of extended header fields in its IDD. Each of these is exposed on the **BlueFile** as an attribute named after the lower case version of the keyword tag. For example, the **DISPLAY_NAME** keyword can be assigned and read via the **.display_name** attribute. **pycs** will make best endeavours to convert any assigned value to the correct type.

A number of keywords are required by RfExtGeo to be accompanied by a similar **.UNITS** suffixed keyword - these will automatically be added if the keyword is set.

Where the keyword has a numbered suffix, this is presented as an integer indexed attribute named after the keyword name without the number suffix or any trailing underscore. For example, **TC_WHOLE_<n>** is represented as an indexed attribute **.tc_whole**, and assigning or reading from **.tc_whole[1]** would set or read the value of keyword **TC_WHOLE_1**. Note that it is the users responsibility to ensure that the 1-up index requirement is met.

Fields which are defined as a number of seconds past the 1950 epoch (e.g. **TC_WHOLE_<n>**) are presented to the user as **datetime** fields, with the usual parsing from strings available, and will be automatically converted to the required representation.

A keyword may be removed by assigning it the value **None** or by calling python's **del** on the attribute. Reading the value of a keyword that is not present will return **None**

Keywords other than those defined by the IDD are prohibited by RfExtGeo, however they may be set and read if necessary by accessing the **.ext_keywords** dictionary, which holds a **(value,type)** tuple for each keyword tag, where type is the Midas letter code defined in table 6 of the Midas Blue specification CCD-2006002.

Convenience attributes are provided for setting groups of attributes that must always appear together, as detailed below.

Sample timestamps

The **.sample_time** attribute allows for setting of the **TCSAMPLE_<m>**, **TC_WHOLE_<m>** and **TC_FRAC_<m>** keywords as a group, as these must always appear together.

The indexed attribute may be assigned or read using an `(tc\_sample, tc\_whole, tc\_frac)` tuple.

```
blue.sample_time[1] = ( 1234, '2021-01-01T00:00:00Z', 0.12345)

(tc_sample,tc_whole,tc_frac) = blue.sample_time[1]
```

is equivalent to

```
blue.tcsample[1] = 1234
blue.tc_whole[1] = '2021-01-01T00:00:00Z'
blue.tc_frac = 0.12345

tc_sample = blue.tcsample[1]
tc_whole = blue.tc_whole[1]
tc_frac = blue.tc_frac[1]
```

Position timestamps

The `.position` attribute allows for setting of the `ETIM<n>`, `POS<xyz><n>`, `VEL<xyz><n>` and `ACC<xyz><n>` keywords as a group, as these must follow strict rules as to which must appear together. The indexed attribute may be assigned or read using an `(etim, (posx, posy, posz), (velx, vely, velz), (accx, accy, accz))` tuple. The `vel` and `acc` arrays/tuples are optional, and if omitted the corresponding keywords will be removed. The `pos`, `vel` and `acc` tuples may also be supplied as arrays. When reading, any missing values will return `None`.

```
blue.position[1] = ( 'now', (1.0, 2.0, 3.0), (4.0, 5.0, 6.0))

(etim, (posx, posy, posz), vel, acc ) = blue.position[1]
```

is equivalent to

```
blue.etim[1] = 'now'
blue.posx[1] = 1.0
blue.posy[1] = 2.0
blue.posz[1] = 3.0
blue.velx[1] = 4.0
blue.vely[1] = 5.0
blue.velz[1] = 6.0
del blue.accx[1]
del blue.accy[1]
del blue.accz[1]

etim = blue.etim[1]
posx = blue.posx[1]
posy = blue.posy[1]
posz = blue.posz[1]
```

```
vel = (blue.velx[1], blue.vely[1], blue.velz[1])
acc = (blue.accx[1], blue.acy[1], blue.accz[1])
```

Covariance Matrix

The `.covariance` attribute allows for population and reading of the `POS_COVAR_MTXxx` and `VEL_COVAR_MTXxy` keywords. Reading the value will return a 6x6 numpy array with the following values populated, where `pxy` is the value of keyword `POS_COVAR_MTXxy`, and `vxy` is the value of `VEL_COVAR_MTXxy`.

	x	y	z	dx	dy	dz
x	p11	p12	p13	0	0	0
y	p12	p22	p23	0	0	0
z	p13	p23	p33	0	0	0
dx	0	0	0	v11	v12	v13
dy	0	0	0	v12	v22	v23
dz	0	0	0	v13	v23	v33

As the matrix is symmetric, when setting the `.covariance` value from a 6x6 array, only the upper triangle is considered, and elements which do not have a corresponding keyword are ignored. It is also possible to set the values using a pair of arrays (or 6-tuples), containing the values (`[p11, p12, p13, p22, p23, p33]`, `[v11, v12, v13, v22, v23, v33]`).

```
blue.covariance = ( [11, 12, 13, 22, 23, 33],
                    [1.1, 1.2, 1.3, 2.2, 2.3, 3.3])
```

is equivalent to

```
import numpy as np
blue.covariance = np.array([
    [11, 12, 13, 0, 0, 0],
    [0, 22, 23, 0, 0, 0],
    [0, 0, 33, 0, 0, 0],
    [0, 0, 0, 1.1, 1.2, 1.3],
    [0, 0, 0, 0, 2.2, 2.3],
    [0, 0, 0, 0, 0, 3.3]])
```

and equivalent to

```
blue.pos_covar_mtx11 = 11
blue.pos_covar_mtx12 = 12
blue.pos_covar_mtx13 = 13
blue.pos_covar_mtx22 = 22
blue.pos_covar_mtx23 = 23
```

```
blue.pos_covar_mtx33 = 33

blue.vel_covar_mtx11 = 11
blue.vel_covar_mtx12 = 12
blue.vel_covar_mtx13 = 13
blue.vel_covar_mtx22 = 22
blue.vel_covar_mtx23 = 23
blue.vel_covar_mtx33 = 33
```

4.2. Reading and Writing data

4.2.1. Writing to File

The `BlueFile` may be written to a file using the `.write()` function, which takes an I/O stream as the first parameter, and an optional `rep` parameter, which should be `IEEE` if the header should be written in big-endian format, or `IIII` for little-endian format. The system endian format will be used by default. The data block will be written in the same endianness as it's source data array.

```
with open('example.midas', 'wb') as f:
    blue.write(f, 'IEEE')
```

4.2.2. Writing to a byte array

The packed data for the file may be accessed as `bytes` using the `.pack()` method, again with an optional `rep` parameter.

```
rawbytes = blue.pack('IEEE')
```

4.2.3. Reading from File

The `BlueFile` may be read from a file using the `.read()` function, which takes an I/O stream as its parameter.

```
with open('example.midas', 'rb') as f:
    blue.read(f)
```

4.2.4. Reading from a byte array

The `MidasFile` can also be populated from a `bytes` array using the `.unpack()` method.

```
blue.unpack(rawbytes)
```

5. Examples

Inside the `pycs.rfextgeo.examples` package are two simple examples of using the `pycs` framework to create a sensor and a tasker.

5.1. Sensor

The example sensor is a simple single sensor client, which registers on the Common Services network and sends out Capability and Status messages as required. It will respond to requests to tune its tuners, start and stop PSD and Audio streams, and to provide I/Q Dwells.

It is not intended as a fully featured sensor simulator, merely as an example of how to use the various frameworks provided by `pycs` from the point of view of a sensor.

The example sensor may be run using the following command line. Once running, all interaction is via messages received over the Common Services network.

```
(venv)$ rfextgeo-example-sensor --help
usage: rfextgeo-example-sensor [-h] [--config Path]
                                [--config-schema | --no-config-schema]
                                [--log-config Path]
                                [--client.registrant-id str]
                                [--client.security.classification {U,R,C,S,TS}]
                                [--client.security.sci-controls str]
                                [--client.security.releaseable-to str]
                                [--client.security.access-controls str]
                                [--client.security.owner-producer str]
                                [--client.security.creator-organization str]
                                [--client.security.classified-by str]
                                [--client.security.classification-reason str]
                                [--client.security.derived-from str]
                                [--client.security.declass-date str]
                                [--client.security.declass-exception str]
                                [--client.security.auth-id str]
                                [--client.security.alt-label str]
                                [--client.signing.key Path]
                                [--client.signing.passphrase str]
                                [--client.signing.certificate Path]
...
...

(venv)$ rfextgeo-example-sensor --config example-sensor-ben-nevis.yml
```

The configuration file should be a YAML file conforming to the schema displayed when supplying the `--config-schema` option

Command line parameters may be used to override values supplied in the configuration file.

The sensor may be configured with a number of different tuners, each able to be tuned independently by a tasker. The sensor will accept any tune requests that are within the configured capability of the tune. Any requests at extremes of the capability may be adjusted to cover the requested frequency without exceeding the capability. Any requests outside the configured capability will be rejected.

The sensor will respond to requests to start and stop PSD and Audio tasks. If the request is accepted, it will stream a dummy PSD or audio stream as appropriate. These dummy streams will be in an appropriate format and with appropriate header values populated, but will contain dummy payloads, a sawtooth pattern for PSD and a constant tone for audio.

The sensor will respond to requests to start and stop an I/Q archive task. Requests for I/Q Observables directed at a running task will be accepted if they conform to the task's parameters, and a dummy I/Q file will be created and made available. This I/Q file will be a valid Midas BLUE file, with appropriate keywords set, and a data block of an appropriate size for the duration and sample rate, but full of zeros.

The sensor does not currently support LOB requests.

Example configuration

```
silkwave:
  hub:
    host: localhost
    port: 61623
    # key_password: pycspass
    # cert_file: client.pem
    # ca_certs: truststore.pem
  client:
    name: pycs-example-sensor-ben-nevis
    security:
      classification: U
      ownerProducer: UK
  client:
    cs_hub: cshub
    registrant_id: urn:csmf:pycs:example:ben-nevis
    security:
      classification: U
      ownerProducer: UK
  sensor:
    display_name: 'Ben Nevis'
    id: urn:csmf:pycs:example:ben-nevis
    classification:
      device:
        classification: U
      receipt:
        classification: U
    origin:
      latitude: 56.7969
      longitude: -5.0036
      height: 1345
```

```
heading: 10
speed: 120
platform:
  display_name: 'Ben Nevis'
  id: urn:csmf:pycs:example:ben-nevis
  type: groundFixed
  pddg: EG
  sigad: EG-123
tuners:
  RX1:
    max_bandwidth: 50MHz
    range:
      min: 100MHz
      max: 400MHz
    center_frequency: 196MHz
    bandwidth: 1MHz
    direct_tasking: true
    tuner_commanding: true
    focal_area:
      latitude: 56.7969
      longitude: -5.0036
      radius: 10000
  RX2:
    max_bandwidth: 50MHz
    range:
      min: 100MHz
      max: 400MHz
    center_frequency: 234MHz
    bandwidth: 40kHz
    direct_tasking: true
    tuner_commanding: true
    focal_area:
      latitude: 56.7969
      longitude: -5.0036
      radius: 10000
  RX3:
    max_bandwidth: 50MHz
    range:
      min: 100MHz
      max: 400MHz
    center_frequency: 345.678MHz
    bandwidth: 20kHz
    direct_tasking: true
    tuner_commanding: true
    focal_area:
      latitude: 56.7969
      longitude: -5.0036
      radius: 10000
  RX4:
    max_bandwidth: 50MHz
    range:
```

```
min: 100MHz
max: 400MHz
center_frequency: 345.678MHz
bandwidth: 20kHz
direct_tasking: true
tuner_commanding: true
focal_area:
  latitude: 56.7969
  longitude: -5.0036
  radius: 10000
```

5.2. Tasker

The example tasker is a simple command line tasker which can query the Common Services network to report on available sensors. It can then tune sensors, playback audio, bring up a plot of PSD, and request I/Q dwell files.

It is not intended as a fully featured tasker, merely as an example of how to use the various frameworks provided by pycs from the point of view of a tasker.

The example sensor may be run using the following command line

```
(venv)$ rfextgeo-example-tasker --help
usage: rfextgeo-example-tasker [-h] [--config Path]
                                [--config-schema | --no-config-schema]
                                [--log-config Path] [--client [JSON]]
                                [--client.security [JSON]]
                                [--client.security.classification {U,R,C,S,TS}]
                                [--client.security.sci-controls str]
                                [--client.security.releaseable-to str]
                                [--client.security.access-controls str]
                                [--client.security.owner-producer str]
                                [--client.security.creator-organization str]
                                [--client.security.classified-by str]
                                [--client.security.classification-reason str]
                                [--client.security.derived-from str]
                                [--client.security.declass-date str]
                                [--client.security.declass-exception str]
                                [--client.security.auth-id str]
                                [--client.security.alt-label str]
                                [--client.signing [JSON]]
                                [--client.signing.key Path]
                                [--client.signing.passphrase str]
...
...

(venv)$ rfextgeo-example-tasker --config example-tasker.yml
>>> sensors
Ben Nevis (pycs:example:ben-nevis)
```

```
Ben Nevis: 56.7969°, -5.0036°, 1345.0m
RX1:
  Range:  [100MHz - 400MHz]
  Current: 196MHz / 1MHz, [195.5MHz - 196.5MHz]
RX2:
  Range:  [100MHz - 400MHz]
  Current: 234MHz / 40kHz, [233.98MHz - 234.02MHz]
RX3:
  Range:  [100MHz - 400MHz]
  Current: 345.678MHz / 20kHz, [345.668MHz - 345.688MHz]
RX4:
  Range:  [100MHz - 400MHz]
  Current: 345.678MHz / 20kHz, [345.668MHz - 345.688MHz]
```

The configuration file should be a YAML file conforming to the schema displayed when supplying the `--config-schema` option

Command line parameters may be used to override values supplied in the configuration file.

On startup the tasker will register on the network as a tasker, and then wait for commands from the user.

5.2.1. Commands

help

Prints a general help message

exit

Exits the tasker

list

List the available commands

sensors

Provides details on all sensors on the network and each of their tuners. Adding the option `--operational` will restrict the list to just those sensors that are available for taksing. Adding the `--tunable` option will restrict the tuner listing to those that support tuner commanding.

tune <sensor-id> <tuner-id> <frequency>

Sends a request to the indicated sensor and tuner to tune to the requested frequency. The frequency can be supplied as Hz, kHz, MHz, GHz or THz.

capability <sensor-id>

Prints a yaml representation of the indicated sensor capability

status <sensor-id>

Prints a yaml representation of the indicated sensor status

audio play <sensor-id> <frequency> <bandwidth> <modulation>

Plays audio from the indicated sensor on the host sound device

audio stop

Stops any playing audio

psd start <sensor-id> <frequency> <bandwidth> [--bins <bins>] [--duration <duration>]

Starts a PSD stream from the indicated sensor and shows a frequency/amplitude plot.

psd stop

Stops any streaming psd

iq start <sensor-id> <frequency> <bandwidth>

Starts an I/Q archive task on the indicated sensor. The task-id will be printed for use in subsequent commands.

iq stop <task-id>

Stops the I/Q task with the indicated id.

iq list

Lists all active I/Q tasks started by this tasker

iq dwell <sensor-id> <frequency> <bandwidth> [--duration <duration>] [--start <start time>] [--task-id <task-id>]

Requests an I/Q dwell file from the indicated sensor. Default duration is 5 seconds, default time is 'now' minus the duration.

ping <resource> [--trace] [--hub] [--hub-only]

Sends a ping command. **--trace** runs a traceroute ping, **--hub** also pings the parent hub, **--hub-only** only pings the parent hub.

5.2.2. Example session

```
INFO:root:Starting...
```

```

Running
Welcome to the CLI interface of tasker!
Try:
* 'help' to display the help message
* 'list' to display the command list.
>>> sensors --tunable
Ben Nevis (pycs:example:ben-nevis)
Ben Nevis: 56.7969°, -5.0036°, 1345.0m
RX1:
  Range: [100MHz - 400MHz]
  Current: 196MHz / 1MHz, [195.5MHz - 196.5MHz]
-----
>>> tune pycs:example:ben-nevis RX1 250MHz
Tuning pycs:example:ben-nevis RX1 to 250MHz
Tuned to 250MHz
>>> sensors --tunable
Ben Nevis (pycs:example:ben-nevis)
Ben Nevis: 56.7969°, -5.0036°, 1345.0m
RX1:
  Range: [100MHz - 400MHz]
  Current: 250MHz / 1MHz, [249.5MHz - 250.5MHz]
-----
>>> exit

```

5.3. Configuration

Both the example sensor and tasker can be configured with a JSON or YAML file conforming to the following schema: The tasker will ignore any **sensor** section, and the sensor will ignore any **tasker** section.

5.3.1. Schema

The schema used to validate config files can be found in the source code at [pycs/rfextgeo/examples/config-schema.yml](#)